Практика 3: OLTP-система и анализ производительности

Table of contents

Практика 3: OLTP-система и анализ производительности (Лекции 5-9) 1
Введение 1
Задачи практики 1
Пример выполнения 3
Задача 1. Модель данных 3
Задача 2. Описание endpoint'а 4
Задача 3. Анализ плана выполнения 5
Задача 4. Влияние индексов 7
Задача 5. Оценка пропускной способности 9

Практика 3: OLTP-система и анализ производительности (Лекции 5-9)

Введение

В реальной разработке необходимо **спроектировать схему данных** и **понять, как СУБД выполняет запросы**. Без этого невозможно выбрать правильные индексы и оценить производительность.

Ключевая идея: прежде чем подключать базу данных к приложению, нужно понять, как она работает изнутри. В этой практике вы:

1. **Спроектируете схему данных** для вашей системы
2. **Увидите, как PostgreSQL выполняет запросы** через EXPLAIN ANALYZE
3. **Сравните поведение разных индексов** (hash vs B-tree)
4. **Оцените пропускную способность** на основе реальных замеров и Little's Law

Контекст: в Практике 2 вы реализовали приложение на Clean Architecture с репозиториями в памяти. Здесь вы впервые работаете с реальной БД — напрямую через psql / DBeaver / DataGrip, без прикладного кода. Цель — понять, как работает хранилище изнутри, прежде чем подключать его к приложению.

Задачи практики

Задача 1. Модель данных в выбранной СУБД

Спроектируйте модель данных в PostgreSQL:

- 4-6 таблиц в формате SQL DDL
- Расчёт размера одной записи в байтах для основных сущностей

- Оценка общего объёма данных при целевой нагрузке

Задача 2. Описание одного эндпоинта из Практики 2

Выберите один эндпоинт с преобладанием записи и детально опишите:

- Название и назначение endpoint'а
- Какие таблицы читаются/пишутся
- SQL-код всех запросов внутри транзакции

Задача 3. Анализ плана выполнения

Создайте таблицы из задачи 1 в PostgreSQL и заполните тестовыми данными (50–100K строк через `generate_series` / `random()`). Запустите `EXPLAIN (ANALYZE, BUFFERS)` для каждого запроса из транзакции задачи 2.

Для каждого запроса зафиксируйте:

- Тип узла (Seq Scan / Index Scan / Index Only Scan)
- Количество строк: estimated vs actual
- Shared buffers: hit (из кэша) и read (с диска)
- Время выполнения (actual time)

Цель: увидеть, как PostgreSQL выполняет ваши запросы, и связать тип узла с лекцией про B-tree, а buffers hit/read — с иерархией памяти.

Задача 4. Влияние индексов

Выберите колонку **без индекса** (например, внешний ключ) и выполните два запроса:

- Точечный запрос: `WHERE col = X`
- Диапазонный запрос: `WHERE col BETWEEN X AND Y`

Сравните три сценария:

1. **Без индекса** — `EXPLAIN (ANALYZE, BUFFERS)` для обоих запросов
2. **Hash-индекс** — `CREATE INDEX ... USING hash(col)`, повторите оба запроса
3. **B-tree-индекс** — `DROP INDEX ...; CREATE INDEX ... USING btree(col)`, повторите оба запроса

Для каждого сценария зафиксируйте: тип узла, buffers hit/read, время.

Вывод: объясните, **почему** hash-индекс ускоряет только point query (хеш-функция теряет порядок ключей), а B-tree — оба типа запросов (упорядоченное дерево позволяет обход диапазона). Свяжите с лекцией про внутреннюю структуру B-tree.

Задача 5. Оценка пропускной способности

Примените Little's Law к реальным замерам из задачи 3:

$$\lambda = L/W$$

Где:

- λ [req/s] — максимальная пропускная способность (max RPS)
- L [req] — размер пула соединений к БД (connection pool size). Задайте реалистичное значение для вашего сценария: типичный диапазон 10–50 для небольшой системы, 50–200 для высоконагруженной
- W [s] — суммарное время транзакции из EXPLAIN ANALYZE (задача 3)

PostgreSQL ограничен числом одновременных соединений, а не ядрами процессора: каждое соединение обрабатывает ровно одну транзакцию в момент времени, поэтому L = размер пула напрямую задаёт параллелизм.

Шаги:

1. Из EXPLAIN ANALYZE (задача 3) возьмите суммарное W транзакции
2. Задайте L — размер пула соединений (обоснуйте выбор)
3. Переведите $\lambda \longrightarrow$ DAU по формуле из лекции «Классификация нагрузки»
4. Напишите 2-3 предложения: какие факторы могут изменить оценку в обе стороны (кэш, пул соединений, конкуренция за блокировки, добавление индексов)

Пример выполнения

Сценарий: интернет-магазин

Задача 1. Модель данных

```sql
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,           -- 8 байт
    email VARCHAR(255) UNIQUE NOT NULL, -- ~30 байт
    password_hash CHAR(60) NOT NULL,    -- 60 байт (bcrypt)
    created_at TIMESTAMP                -- 8 байт
);
-- Размер записи: ~130 байт (с учётом tuple header)

CREATE TABLE products (
    id BIGSERIAL PRIMARY KEY,           -- 8 байт
    name VARCHAR(255) NOT NULL,         -- ~40 байт
    description TEXT,                   -- ~200 байт
    price DECIMAL(10,2) NOT NULL,       -- 8 байт
    stock INT NOT NULL,                 -- 4 байт
    category_id INT                     -- 4 байт
);
-- Размер: ~288 байт

CREATE TABLE orders (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT REFERENCES users(id),
    total DECIMAL(10,2) NOT NULL,
```

```sql
    status VARCHAR(20),
    created_at TIMESTAMP
);
-- Размер: ~76 байт

CREATE TABLE order_items (
    id BIGSERIAL PRIMARY KEY,
    order_id BIGINT REFERENCES orders(id),
    product_id BIGINT REFERENCES products(id),
    quantity INT,
    price DECIMAL(10,2)
);
-- Размер: ~60 байт

CREATE TABLE cart_items (
    user_id BIGINT REFERENCES users(id),
    product_id BIGINT REFERENCES products(id),
    quantity INT,
    added_at TIMESTAMP,
    PRIMARY KEY (user_id, product_id)
);
-- Размер: ~52 байт
```

Оценка объёма:

- 1 млн пользователей × 130 байт = 130 МБ
- 100K товаров × 288 байт = 29 МБ
- 10 млн заказов × 76 байт = 760 МБ
- 30 млн позиций × 60 байт = 1.8 ГБ

Итого: ~2.7 ГБ (без индексов)

Задача 2. Описание endpoint'а

Endpoint: `POST /api/orders` — оформление заказа

Бизнес-логика:

1. Проверить наличие товаров на складе
2. Создать заказ
3. Создать позиции заказа
4. Уменьшить stock
5. Очистить корзину

SQL-код транзакции (3 товара в корзине):

```sql
BEGIN;

SELECT id, stock FROM products WHERE id = $1;
SELECT id, stock FROM products WHERE id = $2;
SELECT id, stock FROM products WHERE id = $3;

INSERT INTO orders (user_id, total, status, created_at)
VALUES ($1, $2, 'pending', CURRENT_TIMESTAMP) RETURNING id;

INSERT INTO order_items (order_id, product_id, quantity, price)
VALUES ($order_id, $p1, $q1, $price1);
INSERT INTO order_items (order_id, product_id, quantity, price)
VALUES ($order_id, $p2, $q2, $price2);
INSERT INTO order_items (order_id, product_id, quantity, price)
VALUES ($order_id, $p3, $q3, $price3);

UPDATE products SET stock = stock - $q1 WHERE id = $p1;
UPDATE products SET stock = stock - $q2 WHERE id = $p2;
UPDATE products SET stock = stock - $q3 WHERE id = $p3;

DELETE FROM cart_items WHERE user_id = $1;

COMMIT;
```

Задача 3. Анализ плана выполнения

Генерация тестовых данных:

```sql
INSERT INTO users (email, password_hash, created_at)
SELECT 'user' || g || '@test.com', md5(g::text),
       NOW() - random() * interval '365 days'
FROM generate_series(1, 100000) AS g;

INSERT INTO products (name, description, price, stock, category_id)
SELECT 'Product ' || g, 'Description ' || g,
       (random() * 1000)::decimal(10,2),
       (random() * 100)::int, (random() * 10 + 1)::int
FROM generate_series(1, 50000) AS g;

INSERT INTO orders (user_id, total, status, created_at)
SELECT (random() * 99999 + 1)::bigint,
       (random() * 500)::decimal(10,2), 'pending',
       NOW() - random() * interval '180 days'
FROM generate_series(1, 100000) AS g;
```

```
INSERT INTO order_items (order_id, product_id, quantity, price)
SELECT (random() * 99999 + 1)::bigint,
       (random() * 49999 + 1)::bigint,
       (random() * 5 + 1)::int,
       (random() * 200)::decimal(10,2)
FROM generate_series(1, 300000) AS g;

INSERT INTO cart_items (user_id, product_id, quantity, added_at)
SELECT (random() * 99999 + 1)::bigint,
       (random() * 49999 + 1)::bigint,
       (random() * 5 + 1)::int,
       NOW() - random() * interval '7 days'
FROM generate_series(1, 50000)
ON CONFLICT DO NOTHING;
```

EXPLAIN для SELECT по PK:

```
EXPLAIN (ANALYZE, BUFFERS) SELECT id, stock FROM products WHERE id = 42;

 Index Scan using products_pkey on products
   (actual time=0.024..0.025 rows=1 loops=1)
   Index Cond: (id = 42)
   Buffers: shared hit=3
 Execution Time: 0.039 ms
```

Index Scan — поиск через B-tree индекс первичного ключа. 3 страницы из кэша (shared hit=3), 0 с диска.

EXPLAIN для UPDATE по PK:

```
EXPLAIN (ANALYZE, BUFFERS)
  UPDATE products SET stock = stock - 1 WHERE id = 42;

 Update on products (actual time=0.052..0.053 rows=0 loops=1)
   ->  Index Scan using products_pkey on products
         (actual time=0.010..0.011 rows=1 loops=1)
         Index Cond: (id = 42)
         Buffers: shared hit=3
   Buffers: shared hit=7 read=1
 Execution Time: 0.068 ms
```

UPDATE тоже использует Index Scan для поиска, затем модифицирует строку in-place. Больше buffers (hit=7, read=1) — запись требует изменения heap-страницы и WAL.

Сводка по транзакции:

Запрос	Узел	Rows	Buffers	Время
3× SELECT products	Index Scan	1	hit=3	~0.04 мс
INSERT orders	Insert	—	hit=4 read=1	~0.10 мс
3× INSERT order_items	Insert	—	hit=3 read=1	~0.08 мс
3× UPDATE products	Index Scan	1	hit=7 read=1	~0.07 мс
DELETE cart_items	Index Scan (PK)	3	hit=5	~0.06 мс
Итого				**~0.9 мс**

Все запросы используют Index Scan — обращения идут по первичному ключу. DELETE по cart_items тоже эффективен: PK (user_id, product_id) — B-tree по составному ключу, поиск по user_id (ведущий столбец) использует индекс.

Задача 4. Влияние индексов

Выбранная колонка: `orders.user_id` — внешний ключ, индекс отсутствует.

1. Без индекса:

```
EXPLAIN (ANALYZE, BUFFERS)
  SELECT * FROM orders WHERE user_id = 42;

 Seq Scan on orders (actual time=0.012..4.523 rows=1 loops=1)
   Filter: (user_id = 42)
   Rows Removed by Filter: 99999
   Buffers: shared hit=935
 Execution Time: 4.561 ms

EXPLAIN (ANALYZE, BUFFERS)
  SELECT * FROM orders WHERE user_id BETWEEN 100 AND 200;

 Seq Scan on orders (actual time=0.015..5.102 rows=101 loops=1)
   Filter: (user_id >= 100 AND user_id <= 200)
   Rows Removed by Filter: 99899
   Buffers: shared hit=935
 Execution Time: 5.134 ms
```

Оба → Seq Scan. Полный перебор 100K строк (935 страниц).

2. Hash-индекс:

```
CREATE INDEX idx_orders_user_id ON orders USING hash(user_id);
```

```
-- Point query → Index Scan (0.044 мс vs 4.561 мс)
 Index Scan using idx_orders_user_id on orders
   (actual time=0.028..0.029 rows=1 loops=1)
   Index Cond: (user_id = 42)
   Buffers: shared hit=2

-- Range query → по-прежнему Seq Scan (5.129 мс)
 Seq Scan on orders (actual time=0.015..5.098 rows=101 loops=1)
   Buffers: shared hit=935
```

Point query ускорен в ~100×. Range query — по-прежнему Seq Scan: хеш-функция теряет порядок ключей, PostgreSQL не может использовать hash-индекс для диапазона.

3. B-tree-индекс:

```
DROP INDEX idx_orders_user_id;
CREATE INDEX idx_orders_user_id ON orders USING btree(user_id);
```

```
-- Point query → Index Scan (0.041 мс)
 Index Scan using idx_orders_user_id on orders
   (actual time=0.025..0.026 rows=1 loops=1)
   Buffers: shared hit=3

-- Range query → Bitmap Index Scan (0.223 мс vs 5.134 мс)
 Bitmap Heap Scan on orders (actual time=0.112..0.198 rows=101 loops=1)
   ->  Bitmap Index Scan on idx_orders_user_id
         (actual time=0.089..0.089 rows=101 loops=1)
         Index Cond: (user_id >= 100 AND user_id <= 200)
   Buffers: shared hit=6
```

Оба типа запросов ускорены. B-tree хранит ключи в отсортированном порядке ⟶ обход диапазона без полного перебора.

Сводка:

Индекс	Point: узел	Point: время	Range: узел	Range: время
Нет	Seq Scan	4.56 мс	Seq Scan	5.13 мс
Hash	Index Scan	0.04 мс	Seq Scan	5.13 мс
B-tree	Index Scan	0.04 мс	Bitmap Scan	0.22 мс

Вывод: hash-индекс $cal(O)(1)$ для equality, но бесполезен для диапазонов — хеш-функция разрушает порядок ключей. B-tree $cal(O)(\log_B n)$ для обоих типов — упорядоченная структура позволяет обход диапазона по связанным листовым узлам.

Задача 5. Оценка пропускной способности

Суммарное время транзакции (из задачи 3): $W \approx 0.9$ мс $= 0.0009$ с.

Пул соединений: PgBouncer, pool_size = 20 (типичный выбор для OLTP-системы среднего размера — меньше нагрузка на PostgreSQL, достаточно для сотен клиентских соединений через мультиплексирование):

$$L = 20$$

Little's Law:

$$\max \text{RPS} = L/W = 20/0.0009 \approx 22222$$

DAU (по формуле из лекции «Классификация нагрузки»):

- Активные часы: 12 ⟶ 43200 секунд
- Действий на пользователя: 20
- Пиковый коэффициент: 2.5

$$\text{DAU} = \frac{22222 \times 43200}{20 \times 2.5} = \frac{959990400}{50} \approx 19200000$$

Результат: ~19 млн DAU

Факторы неопределённости:

- **Снизят:** lock contention на hot rows (UPDATE products), отсутствие индексов на часто фильтруемых полях, long transactions
- **Повысят:** кэширование (Redis) для корзин и каталога, connection pooling (PgBouncer), read replicas для SELECT-запросов